

AI Security Analyst

Comprehensive Project Architecture & Documentation

Introduction

The AI Security Analyst is a full-stack, AI-powered cybersecurity application designed to automate the analysis of system logs. It uses advanced Machine Learning and Large Language Models (LLMs) to identify patterns, detect threats, and provide actionable security recommendations in real-time.

1. Frontend Architecture (React & Vite)

The user interface is built for high performance and optimal user experience:

- **Framework:** React and Vite provide rapid development and fast rendering.
- **Styling:** TailwindCSS is used to implement a modern, responsive, and dark-themed 'Midnight Glass' aesthetic.
- **Key Components:**
 - **LogUpload:** A drag-and-drop interface for users to securely submit `.log` files.
 - **ThreatTimeline:** A chronological visualization dashboard of detected security events.
 - **ChatPanel:** An interactive AI chat interface allowing users to query log data contextually.

2. Backend API (FastAPI)

- **Framework:** Powered by FastAPI, offering high-performance, asynchronous endpoints.
- **Responsibility:** Manages file uploads, triggers log parsing, and coordinates the AI analysis pipelines.
- **Endpoints:** Implements RESTful routes (`/api/logs`, `/api/chat`, `/api/reports`) to communicate seamlessly with the frontend.

3. Log Parsing and Threat Detection

- **Ingestion:** The Log Parser service ingests raw text logs (e.g., Nginx, Apache) and normalizes them into structured data using Regex.
- **Extraction:** Extracts critical fields such as IP Addresses, Timestamps, HTTP Methods, and Status Codes.
- **Detection:** The Threat Detector scans this structured data against known attack signatures, identifying anomalies like SQL Injection, Cross-Site Scripting (XSS), and Path Traversal.

4. AI & Vector Store Integration

- **Vector Search (FAISS):** Processed logs are embedded using HuggingFace sentence transformers and stored in a local FAISS vector database. This allows the system to perform semantic search over millions of log lines instantly.
- **LLM Analysis (LangChain):** The project uses LangChain and Ollama (running local open-source models) to generate human-readable summaries and actionable mitigation steps. Running models locally ensures sensitive log data is not sent to external APIs.

5. Automated PDF Reporting

- **Engine:** Features a built-in generator service utilizing the ReportLab Python library.
- **Output:** Automatically compiles the threat detections, AI summaries, and statistical data into a professional PDF Security Incident Report after each log analysis session.
- **Value:** Provides SOC teams and administrators with ready-to-share, formatted evidence of security postures.